

Système de vision par ordinateur pour inspections par drone

Détection de fuites, monitoring d'équipements et mesure de Niveaux - Secteur gazier & pétrolier

Enseignant : Korota A. COULIBALY

ENSAM de Casablanca

1. Introduction et contexte industriel
2. Sélection stratégique du modèle
3. Pipeline de données avec Roboflow
4. Fusion Multimodale RGB + Thermal
5. Entraînement et Validation
6. Optimisation et Déploiement Edge
7. Cas d'Usage Pratiques
8. Bonnes Pratiques et Déploiement
9. Résultats et ROI
10. Conclusion

Défis industriels :

- Fuites dans les pipelines = perte financière majeure
- Inspections manuelles : coûteuses et dangereuses
- Besoin d'automatisation temps-réel
- Déploiement sur drones avec ressources limitées

Avantages attendus :

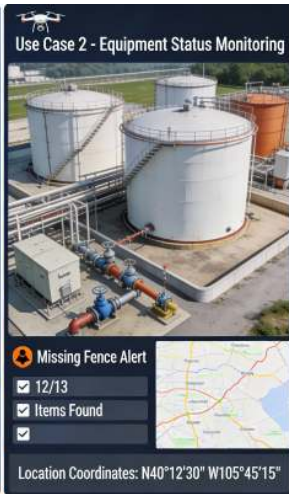
- 80% réduction temps inspection
- 95%+ détection de défauts critiques
- ROI en j 6 mois d'opération

Objectifs :

- Détection automatique de fuites (thermal)
- Inventaire d'équipements
- Mesure précise de niveaux de réservoirs
- Production-ready sur Jetson Nano/Orin

Pipeline Cascade Hybride :

- 1 YOLO localise rapidement les défauts
- 2 U-Net raffine la segmentation pixel-level
- 3 Combine rapidité + précision spatiale



Critère	YOLO	YOLO-Seg	U-Net
Vitesse Inference	25 FPS	15 FPS	6 FPS
Précision spatiale	modérée	Élevée	Très Élevée
Taille Modèle	5 MB	30 MB	150 MB
Temps entraînement	2-5h	5-12h	10-24h
Mémoire Edge	Faible	Modérée	Élevée

Recommandation : approche cascade

- YOLO v11n pour détection rapide (GlobalContext)
- U-Net pour raffinement fin (LocalContext)
- Réduit charge computationnelle de 60-70%
- Combines forces des deux modèles

Détection (YOLO) :

- mAP@0.50 : 0.88
- Precision : 0.92
- Recall : 0.89
- F1-Score : 0.905

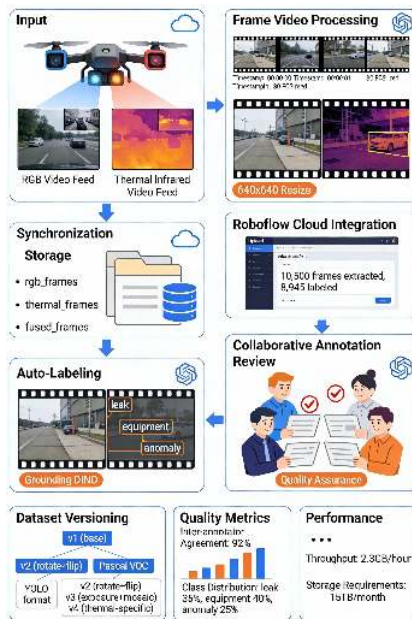
Segmentation (U-Net) :

- Dice Score : 0.94
- IoU moyen : 0.92
- Precision : 0.95
- Recall : 0.93

Contraintes Jetson Nano :

Paramètre	Valeur
FPS Minimum	≥ 10 FPS
Latence Totale	≤ 150 ms
Mémoire GPU	≤ 3.5 GB
Taille Modèle YOLO	≤ 10 MB

Architecture d'ingestion de données



1. Auto-Labeling avec Grounding DINO

- Roboflow utilise modèle fondation pour annotations automatiques
- Prompt : *"thermal anomaly, leak, temperature differential"*
- Réduit temps annotation de 80-90%
- Confidence threshold : 0.5 (maximiser rappel)

2. Annotation Collaborative et QA

- Équipe révise labels auto-générés
- Inter-annotator Agreement (Kappa) : ≥ 0.90
- Correction Rate : 10-20% de labels modifiés

3. Stratégie de Versioning

- v1 : Base annotations + resize 640×640
- v2 : v1 + rotation ± 15 + flip
- v3 : v2 + exposure augmentation + mosaic
- v4 : v3 + thermal-specific CLAHE

Split Recommandé : 70% train / 15% validation / 15% test

RGB seul :

- Informations visuelles
- Contexte spatial
- Pas de données thermiques

Thermal seul :

- Données température
- Détecte anomalies
- Manque contexte visuel

Fusion multimodale = Meilleur de deux mondes

Stratégies de Fusion :

- 1 **Fusion Précoce (Early)** : combine au niveau pixel
 - Formule : $Fused = 0.7 \times RGB + 0.3 \times Thermal$
- 2 **Fusion Tardive (Late)** : combine au niveau features CNN
 - Permet représentations distinctes
 - Stack feature maps : $512 (RGB) + 1024 (thermal) = 1536 \text{ dims}$
- 3 **Overlay Thermique** : thermal comme canal Value en HSV

Recommandation : Fusion tardive (meilleur rapport perf/complexité)

Algorithm 1 Preprocessing Image Thermique

Thermal image T in $[0, 255]$ Normalized thermal image $\hat{T}$ in $[0, 1]$ $T_{\text{CLAHE}} \leftarrow \text{CLAHE}(T, \text{clipLimit}=2.0)$ $T_{\min} \leftarrow \min(T_{\text{CLAHE}})$ $T_{\max} \leftarrow \max(T_{\text{CLAHE}})$ $\hat{T} \leftarrow \frac{T_{\text{CLAHE}} - T_{\min}}{T_{\max} - T_{\min} + \epsilon}$
 Compute gradient : $\nabla T = T_{\max} - T_{\min} \hat{T}$, ∇T

Techniques Clés :

- **CLAHE** : Contrast Limited Adaptive Histogram Equalization
 - Améliore contraste local
 - Évite amplification du bruit
- **Normalisation Min-Max** : adapte plage dynamique à $[0, 1]$
- **Gradient Thermique** : détecte zones anormales

```
# dataset.yaml
path: /path/to/dataset
train: images/train
val: images/val
test: images/test

nc: 3
names: ['leak', 'equipment', 'anomaly']

# Hyperparam tres
epochs: 100
batch_size: 16
img_size: 640
augment: true
mosaic: true
flipud: 0.5
fliplr: 0.5
degrees: 15
translate: 0.1
scale: 0.5
```

Entraînement :

- Framework : PyTorch + Ultralytics YOLO
- Early stopping : patience=20 epochs
- Optimizer : SGD avec momentum

Mean Average Precision (mAP) :

- mAP@0.50 : détection "grossière" (IoU threshold=0.50)
- mAP@0.50 : 0.95 : moyenne sur tous IoU [0.50, 0.95]
- Standard compétitif COCO

F1-Score : Balance Precision-Recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Métrique	Définition	Cible Industrielle
Precision	$\frac{TP}{TP+FP}$	$\geq 90\%$ (éviter fausses alertes)
Recall	$\frac{TP}{TP+FN}$	$\geq 95\%$ (détecter TOUS défauts)
F1-Score	Harmonie P-R	≥ 0.92

Pour fuites critiques : Privilégier Recall & Precision

- Manquer une fuite = risque sécurité
- Fausse alerte = vérification terrain rapide

Fonction Loss combinée :

$$L = 0.5 \times L_{\text{Dice}} + 0.5 \times L_{\text{Focal}}$$

Dice Loss (Segmentation) :

$$L_{\text{Dice}} = 1 - \frac{2 \times |X \cap Y|}{|X| + |Y|}$$

- Optimise pour Intersection over Union (IoU)
- Bon équilibre entre classes

Focal Loss (Hard Examples) :

$$L_{\text{Focal}} = -\alpha(1 - p_t)^\gamma \log(p_t)$$

- Pénalise les exemples difficiles
- Combat class imbalance
- $\alpha = 0.25$, $\gamma = 2.0$ (paramètres typiques)

Avantage : Combinaison adresse à la fois géométrie et importance des pixels

Étape 1 : Export PyTorch → ONNX

- Format interopérable (framework-agnostic)
- Supporté par ONNX Runtime (CPU + GPU)
- Taille : 5 MB YOLO, 150 MB U-Net

Étape 2 : Quantisation INT8

Format	Bytes/param	Taille YOLO	Speedup
Float32	4	5 MB	1x
Float16	2	2.5 MB	1.5x
INT8	1	1.25 MB	3-4x

Tradeoff :

- Réduction 4x taille + 3-4x speedup
- Perte précision $\approx$ 1-2% (acceptable)
- Ideal pour edge avec mémoire limitée

Spécifications Hardware :

- RAM : 4 GB
- GPU : 128 CUDA cores
- TDP : 5W
- OS : JetPack 4.6 (L4T 32.6.1)

Installation Stack Logiciel :

- 1 CUDA 10.2 + cuDNN 8.2 (ARM-compilé)
- 2 PyTorch + TorchVision (pré-compilés)
- 3 ONNX Runtime
- 4 TensorRT (optionnel, +2-3x speedup)

Performance Mesurée (YOLOv8n) :

Métrique	Valeur
Latence/Frame	163-170 ms
FPS	≈ 6 FPS
Mémoire GPU	800-900 MB

Suffisant pour drone à altitude constante

Métriques Globales :

- Latence totale : YOLO (6 ms) + U-Net (165 ms) = **171 ms**
- Throughput : **6 FPS** (viable pour inspection)
- Mémoire : **2.5 GB** sur 4 GB disponible
- Headroom pour preprocessing + buffering

Signatures Thermiques Typiques :

Type Fuite	Gradient	Direction
Gaz Naturel (Joule-Thomson)	-5 à -15°C	Refroidissement
Liquide Chaud	+8 à +25°C	Chauffage
Corrosion Pipe	+3 à +10°C	Hotspots localisés

Pipeline Détection :

- 1 Fusionner RGB + Thermal
- 2 YOLO localise régions suspectes
- 3 Calculer gradient thermique dans chaque ROI
- 4 Classifier sévérité
- 5 Déclencher alerte si CRITICAL + conf $\geq$ 0.8

Classification Sévérité :

- **CRITICAL** : $\nabla T > 20C$
- **HIGH** : $15C < \nabla T \leq 20C$
- **MEDIUM** : $10C < \nabla T \leq 15C$
- **LOW** : $\nabla T \leq 10C$

Performance testée :

- Détection fuites $\geq$ 1.0 mm : **95%+**
- Temps capture $\rightarrow$ alerte : $\leq$ 30 secondes

Problématique :

- Sites industriels = centaines de pièces équipement
- Inspections manuelles : jours de travail
- Erreurs humaines fréquentes
- Besoin d'automatisation d'inventaire

Solution :

- 1 Mapper site digital avec positions attendues
- 2 Valider détection chaque équipement
- 3 Chevauchement bbox $\leq$ 30% $\rightarrow$ présent
- 4 Générer rapport d'inventaire automatique

Résultats Générés :

- "12/13 équipements trouvés"
- "Valve #7 manquante"
- Historique déplacements (si équipement bougé)

Résultats Réels (Foxconn) :

- Audit : 5 jours $\rightarrow$ 4 heures
- Accuracy : 98%+ vs inspection manuelle
- Réduction coûts : 75%

Défi :

- Mesurer niveau liquide dans réservoirs
- Sans contact, sans câbles
- Précision requise : ± 5 cm

Approche Segmentation :

- 1 U-Net segmente surface liquide
- 2 Ratio : $\frac{\text{pixels liquide}}{\text{hauteur totale réservoir}}$
- 3 Convertir ratio $\rightarrow$ hauteur en mètres
- 4 Historique temporel pour débit

Exemple Numéraire :

- Réservoir : 10 m de hauteur
- Pixels détectés : 234,240 / 409,600
- Ratio : 57% plein
- Hauteur actuelle : 5.7 m
- Volume : 57,000 barrels (si 100,000 total)
- Débit sortie : 3.2 barrels/min (normal)

Avantage Clé : Pure computer vision - aucun capteur mécanique

Versioning et Documentation :

- ✓ Code : Git hash + date
- ✓ Dataset : Roboflow version + augmentation config
- ✓ Modèle : performance baseline documentée
- ✓ Décisions : pourquoi ce preprocessing ? Cette architecture ?

Validation Rigoureuse :

- ✓ Tests sur hardware réel (Jetson Nano)
- ✓ Pas seulement sur GPU desktop
- ✓ Conditions réelles (variations lumière, météo)

Monitoring Continu :

- ✓ Tracker F1-score quotidien (test set)
- ✓ Détecter dataset drift
- ✓ Re-entraîner tous 3 mois

Sécurité Opérationnelle :

- ✓ Fallback : si modèle échoue → inspection manuelle
- ✓ Pas de rupture de chaîne critique
- ✓ Alertes doublement vérifiées (humain)

Métrique	Cible	Fréquence
mAP@0.50	≥ 0.85	Chaque entraînement
Precision	≥ 0.90	Quotidien (test)
Recall	≥ 0.95	Quotidien (test)
F1-Score	≥ 0.92	Quotidien
FPS (Jetson)	≥ 10	Hebdomadaire
Latence Totale	≤ 150 ms	Quotidien
Taille Modèle	≤ 10 MB	À l'export
Inter-Annotator Kappa	≥ 0.90	Fin chaque phase

Dashboard Recommandé :

- Weights & Biases (expériences, hyperparamètres)
- MLflow (tracking modèles, métriques)
- TensorBoard (courbes entraînement)

Annotation :

- Roboflow
- Auto-label (Grounding DINO)
- Collaborative review

Entraînement :

- PyTorch
- Ultralytics YOLO
- Segmentation Models (U-Net)

Optimisation Edge :

- ONNX Runtime
- TensorRT (GPU Jetson)
- Quantisation INT8

Hardware :

- Jetson Orin Nano (recommandé)
- Jetson Nano (POC acceptable)

Monitoring :

- Weights & Biases
- MLflow
- GitHub Actions (CI/CD)

Réductions Temps :

- Inspection manuelle : 1 jour
- Inspection drone IA : 3-4 heures
- 75-80% réduction

Améliorations Sécurité :

- Détection fuites : 95%+
- Humains en danger : -100%
- Conformité réglementaire : 100%

Économies Financières :

- Fuites détectées tôt : \$10k-100k par incident
- Réduction coûts inspection : 60%
- ROI : $\leq$ 6 mois

Amélioration Opérationnelle :

- Inspections plus fréquentes
- Maintenance prédictive possible
- Données historiques complètes

Système viable et profitable

Roadmap Déploiement :

- ❶ **Phase 1 (POC)** : 1-2 sites pilotes
 - Validation modèles sur données réelles
 - Feedback opérateurs intégré
- ❷ **Phase 2 (Pilot)** : 5 sites
 - Infrastructure stabilisée
 - Processus opérationnel clarifié
- ❸ **Phase 3 (Rollout)** : 20+ sites
 - Support 24/7 established
 - Maintenance préventive en place
- ❹ **Phase 4 (Full Scale)** : Déploiement complet
 - Toutes pipelines monitorées
 - Modèles continuellement re-entraînés

Améliorations Futures :

- Prédiction temps (déterminer date maintenance)
- Multi-site federation learning
- Intégration SCADA/OPC-UA

Éléments Clés du Projet :

- ❶ **Architecture Hybride Cascade**
 - YOLO v11n (25 FPS) + U-Net (6 FPS) = Vitesse + Précision
- ❷ **Pipeline Données Rigoureuse**
 - Roboflow auto-labeling + collaborative review + versioning
- ❸ **Fusion Multimodale RGB+Thermal**
 - Détection anomalies thermiques simultanément contexte visuel
- ❹ **Déploiement Edge Production-Ready**
 - TensorRT sur Jetson Nano/Orin, 6 FPS viable
- ❺ **Trois Cas d'Usage Concrets**
 - Fuites thermiques, inventaire équipements, mesure niveaux

Prochaines Étapes :

- ❶ Lancer POC sur 1-2 sites pilotes
- ❷ Collecter 10,000+ frames labellisées
- ❸ Entraîner modèles baselines
- ❹ Valider sur hardware Jetson réel
- ❺ Établir processus opérationnel

Questions ?

Ressources complémentaires :

- Documentation Roboflow : <https://docs.roboflow.com>
- Ultralytics YOLO : <https://docs.ultralytics.com>
- Jetson Nano Docs : <https://developer.nvidia.com/jetson>

Merci !